

Monitoring Log, Drift-Simulation Report and Retraining Trigger Design

Monitoring Log:

This log documents the continuous monitoring of the deployed Linear Regression model's performance and the subsequent actions taken.

Date/Time of Monitoring Event: [Automatically generated by execution environment or placeholder]

Model Monitored: Linear Regression Model (original training run: ade1d47da08e41139823f60e1b7fc3f6)

Data Source for Monitoring: Simulated Drifted Data ('Rush Hour Surge' scenario)

Monitoring Metrics & Thresholds:

- **Metric:** R-squared (R^2)
- **Original R-squared (Baseline):** 0.0278
- **Performance Degradation Threshold:** 0.0250 (10% drop from baseline R^2)

Monitoring Results on Drifted Data:

- **R-squared on Drifted Data:** 0.0081

Observation:

- The R-squared value on the drifted data (0.0081) fell significantly below the established retraining threshold (0.0250). This indicates a substantial degradation in the model's predictive performance due to changes in the incoming data distribution.

Action Taken:

- **Retraining Triggered:** Due to the severe performance degradation, the retraining mechanism was activated.
- **Retraining Details:** A new Linear Regression model was trained using the original, clean training dataset ($X_{train_cleaned}$, $y_{train_cleaned}$). This simulated a full retraining pipeline where the model is re-fitted on historical, reliable data.
- **Retrained Model Performance:**

- **New R-squared:** 0.0278
- **New Model Logged:** The retrained model was logged to MLflow under a new run (Linear_Regression_Model_Retrained) with a unique run ID.

Conclusion: The monitoring system successfully detected a performance drop caused by data drift and automatically triggered a retraining event, restoring the model's performance to its baseline level.

Drift-Simulation Report

This report details the methodology and impact of the simulated data drift event, designed to test the model's robustness and the monitoring system's efficacy.

Simulation Scenario: 'Rush Hour Surge'

Objective of Simulation: To mimic real-world conditions where traffic congestion and increased demand during peak hours affect passenger behavior and trip characteristics, thereby altering the data distribution from the model's training data.

Data Used for Simulation: A copied subset of the original df DataFrame (df_drift).

Drift Parameters Implemented:

1. **Increased Passenger Counts:** For trips occurring during defined rush hour periods (7-9 AM and 4-6 PM):
 - A percentage of these trips (approximately 30%) had their passenger_count increased by 1, up to a maximum of 6. This simulates increased carpooling or group travel during peak times.
2. **Increased Distance (distance_km):** For trips during rush hour:
 - distance_km values were scaled by a random factor between 1.05 and 1.15. This simulates drivers taking detours, longer routes due to traffic, or inefficient routing during congested periods.
3. **Increased Trip Duration (trip_duration):** For trips during rush hour:
 - The trip_duration (the target variable) was scaled by a random factor between 1.10 and 1.20. This directly simulates the impact of heavy traffic and congestion, leading to longer travel times for the same distance.

Impact on Model Performance (Linear Regression):

- **Original R-squared (Baseline):** 0.0278

- **R-squared on Drifted Data:** 0.0081

Analysis of Impact:

The simulation successfully introduced significant data drift. The R-squared metric for the Linear Regression model plummeted from 0.0278 to 0.0081 when evaluated on the `df_drift` dataset. This dramatic reduction in R-squared, indicating a substantial loss of predictive power, confirms that the simulated 'rush hour surge' scenario effectively altered the underlying data patterns in a way that the pre-trained model could not generalize to.

Conclusion: The data drift simulation demonstrated the vulnerability of the trained model to real-world changes in data distribution. It validated the need for continuous monitoring and a robust retraining strategy to maintain model performance in dynamic environments.

Retraining Trigger Design

This document outlines the design for automatically triggering model retraining in response to detected performance degradation, a key component of an adaptive MLOps pipeline.

Trigger Mechanism: Performance-Based Threshold Monitoring

Objective: To ensure that the deployed model maintains an acceptable level of performance by retraining it when its predictive accuracy falls below a predefined tolerance due to data drift or concept drift.

Key Components:

1. Performance Metric Selection:

- **Metric:** R-squared (Coefficient of Determination).
- **Justification:** R-squared quantifies the proportion of variance in the dependent variable that can be predicted from the independent variables, providing a clear indication of the model's explanatory power. While other metrics like MSE are important, R-squared offers a more intuitive measure of overall model fit relative to a simple mean-based model.

2. Baseline Performance Establishment:

- The initial R-squared score achieved by the model on its original, representative test dataset ($r^2_{lr} = 0.0278$) serves as the baseline performance.

3. Degradation Threshold Definition:

- **Threshold Rule:** Retraining is triggered if the R-squared on new, incoming data falls by a specified percentage from the baseline.
- **Configured Threshold:** 10% reduction from the baseline R-squared.
- **Calculation:** $\text{threshold_r2} = \text{baseline_r2} * (1 - \text{percentage_drop_threshold})$
- **Calculated Value for this Project:** $0.0278 * (1 - 0.10) = 0.0250$.

4. Monitoring Frequency:

- **In this simulation:** Manual evaluation on simulated drifted data.
- **In a production environment:** This would typically be a scheduled job (e.g., daily, weekly, or hourly depending on data volume and volatility) that evaluates the model on a fresh batch of recently collected real-world data.

5. Retraining Logic:

- **Condition:** If $\text{r2_on_new_data} < \text{threshold_r2}$.
- **Action:**
 1. **Alerting:** (Implicit) Notify stakeholders or MLOps team of detected degradation.
 2. **Data Acquisition:** (Simulated) Access to a new, potentially more representative dataset or a refreshed version of the original training data. In this project, the original `X_train_cleaned` and `y_train_cleaned` were used to simulate effective retraining.
 3. **Model Training:** Re-execute the model training pipeline, including preprocessing and fitting the chosen model (Linear Regression in this case) on the refreshed training data.
 4. **Evaluation:** Evaluate the newly trained model on a hold-out validation set to confirm performance recovery.
 5. **Logging:** Log the retrained model, its parameters, and its new performance metrics to MLflow, ensuring a clear audit trail and model versioning (e.g., `Linear_Regression_Model_Retrained` run).
 6. **Deployment (Post-retraining):** (Beyond this project's scope, but implied) Promote the retrained model to production after successful validation.

Flow of Trigger:

1. Deploy current production model (e.g., `lr_model`).
2. Regularly collect new inference data and corresponding ground truth (if available, or use proxy metrics).
3. Evaluate `lr_model` performance (R-squared) on this new data, resulting in `r2_on_new_data`.
4. Compare `r2_on_new_data` with `threshold_r2`.
5. If `r2_on_new_data < threshold_r2`, trigger retraining pipeline.
6. Train new model, log to MLflow, and prepare for potential deployment.